

CYBERSECURITY & ETHICAL HACKING

Task 6: Full Penetration Testing & Red Team Simulation Project

Penetration Testing Report

Authorized OWASP Juice Shop Local-Lab Assessment

Submitted by	Rahul Kumar
Branch	Electronics & Computer Science Engineering
College	Raj Kumar Goel Institute of Technology
Task	Task 6 — Full Penetration Testing & Red Team Simulation
Assessment date	09 September 2026
Target	OWASP Juice Shop — isolated local training lab

Academic integrity and disclosure: This is a polished academic sample report. All evidence panels in this document are clearly labelled illustrative/recreated because they were generated for document presentation, rather than captured from a live assessment. The target, IP address, outputs, users, and findings are fictional teaching examples. Do not claim that this report is a real assessment of any organization or public website.

Executive Summary

This report presents a controlled penetration-testing simulation against OWASP Juice Shop, an intentionally vulnerable application deployed in an isolated local training environment. The objective was to follow the complete assessment lifecycle: reconnaissance, service scanning, controlled web-application testing, impact evaluation, evidence collection, reporting, and cleanup.

The simulated assessment identified four representative security findings: weak authentication input handling, reflected cross-site scripting, inadequate object-level authorization, and verbose error behavior. The highest-priority issues were authentication input handling and missing authorization checks because, in a real application, they may enable unauthorized access to accounts or user data. No destructive actions, persistence, denial-of-service tests, real data collection, or testing outside the local lab were performed.

Scope and Rules of Engagement

Scope item	Details
In scope	One intentionally vulnerable OWASP Juice Shop instance at fictional local IP 192.168.56.110:3000.
Testing workstation	Kali Linux virtual machine on a host-only/local virtual network.
Allowed testing	Discovery, safe enumeration, browser and proxy inspection, and controlled challenge validation.
Out of scope	Public systems, external networks, denial-of-service, persistence, destructive actions, phishing, and real data exfiltration.
Safety condition	All testing was limited to synthetic data in the educational lab.

1. Introduction

Penetration testing is an authorized security assessment that evaluates how weaknesses could be used by an attacker. In an educational environment, intentionally vulnerable applications allow students to understand security risks without harming real systems. OWASP Juice Shop was selected as the training target because it contains safe, documented challenges involving authentication, input validation, authorization, and security configuration.

Objectives

- Identify exposed network services on the authorized target.
- Observe web requests and application functionality.
- Validate representative weaknesses using non-destructive methods.
- Assess likely impact and prioritize remediation.
- Create a professional report with clear evidence and recommendations.

2. Methodology

Reconnaissance

The local target address and application URL were documented. The application landing page, login area, search functionality, and visible routes were reviewed.

Scanning and enumeration

Nmap was used to identify the exposed service and obtain service information. Browser inspection and a web proxy were used to understand normal request patterns.

Controlled validation

Each finding was simulated with harmless test data. Testing stopped after sufficient evidence of the security effect was obtained.

Analysis and reporting

Findings were rated according to the likely ease of exploitation and the potential effect on confidentiality, integrity, and availability.

3. Reconnaissance and Scanning

The authorized target was identified as a local OWASP Juice Shop instance. The evidence panel below is an illustrative recreation of a service-discovery result, included to demonstrate professional report formatting.

Figure 1 — Nmap Service Discovery (Illustrative)

```
root@kali:~# nmap -sV -p 3000 192.168.56.110
Starting Nmap 7.94 ( https://nmap.org )
Nmap scan report for 192.168.56.110
Host is up (0.0012s latency).

PORT      STATE SERVICE VERSION
3000/tcp  open  http   Node.js Express framework

Service detection performed. Scan completed successfully.
```

ILLUSTRATIVE / RECREATED LAB EVIDENCE — generated for academic-report presentation; not a live screenshot or proof of testing a real system.

Interpretation: HTTP was observed on TCP/3000, with a Node.js/Express service fingerprint. The application was accessed only through the fictional local virtual network.

Figure 2 — Authorized Lab Target (Illustrative)

```
OWASP Juice Shop – Local Training Instance
URL: http://192.168.56.110:3000

[ Home ] [ Products ] [ Login ] [ Basket ]

Search products: [ _____ ] [ Search ]

This deliberately vulnerable application is used only in an isolated lab.
No production or external service was accessed.
```

ILLUSTRATIVE / RECREATED LAB EVIDENCE — generated for academic-report presentation; not a live screenshot or proof of testing a real system.

4. Application Request Review

A web proxy is useful for understanding how a browser communicates with the approved application. In this simulation, normal product-search, login, and basket-related requests were reviewed before controlled testing was attempted.

Figure 3 — Sanitized Request Observation (Illustrative)

```
Burp Suite Community Edition — Proxy / HTTP history

Method URL Status Length
GET /rest/products/search?q=apple 200 8214
POST /rest/user/login 200 612
GET /rest/basket/2 200 1320

Requests shown are representative, sanitized lab examples.
```

ILLUSTRATIVE / RECREATED LAB EVIDENCE — generated for academic-report presentation; not a live screenshot or proof of testing a real system.

In an actual assessment, screenshots should be captured directly by the student and sensitive values must be redacted before submission or public sharing.

5. Technical Findings

F-01: Weak authentication input handling

Severity: High

Description: The training login workflow accepted crafted input in a way that altered expected authentication behavior. A safe simulation showed that query manipulation could bypass normal password verification.

Impact: A real attacker might gain unauthorized account access, exposing protected data or privileged functions.

Recommendation: Use parameterized queries throughout the application, validate input server-side, apply generic error handling, and use least-privileged database accounts.

F-02: Reflected cross-site scripting (XSS)

Severity: Medium

Description: A search-related parameter was reflected into an application page without sufficient context-aware output encoding in the educational scenario. A harmless proof demonstrated client-side execution.

Impact: A real attacker could create a link that executes script in a victim's browser and may enable deceptive content or unauthorized in-session actions.

Recommendation: Encode output according to its HTML/JavaScript/URL context, validate input, use safe templates, deploy Content Security Policy, and secure session cookies.

F-03: Inadequate object-level authorization

Severity: High

Description: A synthetic resource belonging to another test user was reachable after modifying an object identifier in an approved training endpoint. The server-side access check was insufficient.

Impact: A real user might read or change another user's records, creating privacy, fraud, and compliance risks.

Recommendation: Enforce authorization on every server-side object request; bind access to the authenticated identity; add negative authorization tests and security logging.

F-04: Verbose error response

Severity: Low

Description: A malformed training request returned implementation detail that could help an attacker identify the technology stack. No secrets were exposed in the simulation.

Impact: Information disclosure can reduce reconnaissance effort and make later attacks more efficient.

Recommendation: Show generic client errors, keep detailed diagnostics on the server, disable debug mode in production, and review error responses.

6. Finding Evidence

The following panel presents the format of controlled evidence used for the object-level authorization finding. It intentionally contains no real credentials, tokens, customer records, or exploit payloads.

Figure 4 — Controlled Finding Evidence (Illustrative)

```
Controlled validation record

Finding ID: F-03 — Object-level authorization weakness
Test account: student_test (synthetic lab account)
Result: a modified object identifier returned a synthetic record
Impact proof: unauthorized read access was demonstrated safely

No real customer data, passwords, tokens, or external systems were involved.
```

ILLUSTRATIVE / RECREATED LAB EVIDENCE — generated for academic-report presentation; not a live screenshot or proof of testing a real system.

Evidence handling: Screenshots and outputs must be stored securely. When publishing a portfolio or LinkedIn post, redact IP addresses, usernames, tokens, headers, payloads, personal data, and any detail that could enable misuse.

7. Risk Summary

Finding	Severity	Risk rationale
F-01: Authentication input handling	High	High potential impact because unauthorized account access may follow.
F-02: Reflected XSS	Medium	Requires user interaction but can affect browser-based sessions.
F-03: Object-level authorization	High	May expose or modify data belonging to another user.
F-04: Verbose errors	Low	Limited direct impact but assists reconnaissance.

Overall risk: **High** for a production-equivalent application because two findings could facilitate unauthorized access to accounts or records. In this classroom scenario, the purpose is learning and remediation rather than attacking a real service.

8. Remediation Roadmap

Priority	Recommended action	Suggested timeframe
1	Replace unsafe query construction with parameterized queries; add login-bypass regression tests.	Immediate / 7 days
2	Implement centralized object-level authorization and add ownership test cases.	Immediate / 14 days
3	Apply context-aware output encoding, CSP, and secure cookie settings.	30 days
4	Remove verbose error detail and improve server-side logging and monitoring.	30 days
5	Conduct a retest and record closure evidence for each finding.	After remediation

9. Conclusion

The Task 6 simulation demonstrated the complete penetration-testing lifecycle in a safe educational environment. Starting from service discovery, the assessment moved through request observation, controlled vulnerability validation, impact analysis, and remediation planning. The report shows that secure query design, server-side authorization, safe output handling, and production-hardening practices are essential for reducing common web-application risks.

10. Cleanup Statement

The lab exercise did not create persistent access, modify system configuration, perform denial-of-service testing, access external systems, or extract real data. All work was limited to an illustrative local training scenario. The target environment was considered restored after the exercise, and any temporary test sessions were cleared.

Prepared by Rahul Kumar | Electronics & Computer Science Engineering | Raj Kumar Goel
Institute of Technology